

LAPORAN TUGAS BESAR

IF2224 Teori Bahasa Formal dan Automata

Semester II 2025/2026

Arion Compiler Milestone 4

Intermediate Code and Interpreter



Stevanus Agustaf Wongso 13524020

Renuno Yuqa Frinardi 13524080

Valentino Daniel Kusumo 13524104

Michael James Liman 13524106

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

2026

Daftar Isi

1	Landasan Teori	1
1.1	Arsitektur Kompiler: Front-end dan Back-end	1
1.2	Intermediate Code Generation	1
1.2.1	Decorated AST sebagai Sumber Kebenaran	1
1.2.2	Three-Address Code (TAC)	2
1.2.3	Translasi Control Flow	2
1.3	Katalog Instruksi Intermediate Code	3
1.4	Arsitektur Mesin Eksekusi: Interpreter dan Stack	3
1.4.1	Virtual Machine dan Siklus Eksekusi	3
1.4.2	Memori Eksekusi: The Stack	4
1.5	Keamanan Runtime dan Penanganan Kerentanan	4
2	Perancangan dan Implementasi	5
2.1	Perancangan	5
2.1.1	Gambaran Umum Sistem	5
2.1.2	Teknologi yang Digunakan	5
2.1.3	Struktur Modul dan Arsitektur File	5
2.1.4	Justifikasi Pilihan Implementasi	6
2.2	Implementasi Intermediate Code Generator	6
2.2.1	Pemetaan Alamat Memori	7
2.2.2	Pembentukan Ekspresi (DFS Postorder)	7
2.2.3	Pembentukan Control Flow	8
2.2.4	Resolusi Label dan Pencetakan	8
2.3	Implementasi Interpreter (Stack Machine)	8
2.3.1	Representasi Nilai dan Konteks	8
2.3.2	Siklus Fetch–Decode–Execute	8
2.3.3	Inisiasi Frame dan Akses Memori	9
2.3.4	Lompatan dan Konvensi IF_FALSE	9
2.3.5	Operasi melalui OPR	10
2.4	Implementasi Error Handling dan Keamanan Runtime	10
3	Pengujian	11
3.1	Kasus 1 – Penugasan dan Cetak Literal	11
3.2	Kasus 2 – Ekspresi Aritmatika dan Relasional	12
3.3	Kasus 3 – Ekspresi Bersarang dan Modulus	13
3.4	Kasus 4 – Kontrol Alur IF-ELSE dan WHILE	14
3.5	Kasus 5 – Penanganan Error: Pembagian dengan Nol	15
3.6	Kasus 6 – Penanganan Error: Numerical Overflow	16
3.7	Ringkasan Pengujian	17
4	Kesimpulan dan Saran	17

4.1	Kesimpulan	17
4.2	Saran	18
5	Lampiran	18
5.1	Tautan Release Repository GitHub	18
5.2	Pembagian Tugas	18
6	Referensi	19

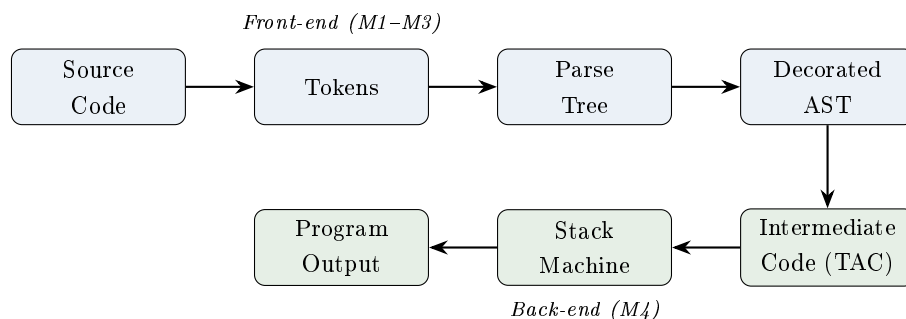
1. Landasan Teori

Milestone 4 merupakan tahap akhir dari pengembangan kompiler bahasa Arion. Pada tahap ini, *Decorated AST* yang dihasilkan Milestone 3 diubah menjadi *Intermediate Code* berbentuk *Three-Address Code* (TAC), lalu dieksekusi oleh sebuah *interpreter* berbasis *stack machine* hingga menghasilkan keluaran nyata program. Bab ini menjelaskan teori dan konsep yang melandasi kedua komponen tersebut.

1.1 Arsitektur Kompiler: Front-end dan Back-end

Kompiler modern umumnya dibagi menjadi dua fase arsitektur: **front-end** dan **back-end**. Milestone 1 sampai 3 (analisis leksikal, sintaksis, dan semantik) merupakan bagian front-end yang berurusan langsung dengan teks kode sumber—memotongnya menjadi token, memvalidasi tata bahasanya, lalu memastikan maknanya benar. Keluaran front-end adalah struktur data murni berupa *Decorated AST* yang sudah bebas dari teks mentah.

Milestone 4 sepenuhnya berada pada area back-end. Fase ini tidak lagi peduli pada detail sintaksis Arion (titik koma, kurung, atau spasi), melainkan berfokus pada dua tugas inti: **sintesis**, yaitu mengonversi *Decorated AST* menjadi instruksi operasional standar (*Intermediate Code*), dan **eksekusi**, yaitu mengelola memori serta menjalankan instruksi tersebut pada mesin virtual. Pemisahan yang tegas ini memberi keuntungan rekayasa: bila suatu saat sintaks Arion diubah, cukup front-end yang dirombak selama *Decorated AST* yang dihasilkan tetap sama, back-end tidak perlu disentuh.



Gambar 1: Pipeline kompiler Arion. Milestone 4 menambahkan tahap pembentukan *Intermediate Code* dan eksekusi pada *stack machine*.

1.2 Intermediate Code Generation

1.2.1 Decorated AST sebagai Sumber Kebenaran

Decorated AST adalah *Abstract Syntax Tree* yang setiap simpulnya telah dianotasi (didekorasi) dengan informasi semantik, terutama **tipe data** hasil ekspresi dan **referensi tabel simbol** untuk variabel. Anotasi ini bersifat wajib bagi generator kode. Sebagai ilustrasi, ketika generator menemui ekspresi `a + b`, tanpa informasi tipe ia tidak dapat memutuskan apakah operasi tersebut penjumlahan bilangan atau penggabungan string. Dengan *Decorated AST*, generator dapat memilih instruksi *Intermediate Code* yang tepat.

1.2.2 Three-Address Code (TAC)

Langkah berikutnya adalah meratakan (*flattening*) Decorated AST menjadi daftar instruksi linier. Format yang digunakan adalah **Three-Address Code**, dinamai demikian karena tiap instruksi maksimal melibatkan tiga alamat (dua operand sumber dan satu tujuan), dengan struktur dasar `hasil = operand1 operator operand2`.

Karena CPU tidak dapat menghitung ekspresi panjang sekaligus, TAC memecah ekspresi kompleks menggunakan **variabel sementara** (*temporary*, ditulis `t1`, `t2`, ...). Pohon ekspresi dievaluasi dari cabang terdalam menuju akar. Misalnya, ekspresi `x := (a + b) * (c - d)` diratakan menjadi:

```
t1 := a + b      { evaluasi tanda kurung pertama }
t2 := c - d      { evaluasi tanda kurung kedua   }
t3 := t1 * t2    { kalikan hasil keduanya       }
x  := t3         { simpan hasil akhir ke x       }
```

Pada implementasi berbasis stack machine, peran variabel sementara digantikan oleh puncak stack: hasil antara cukup ditinggalkan di atas stack untuk dikonsumsi operasi berikutnya, sehingga tidak perlu dialokasikan nama `t1`, `t2` secara eksplisit.

1.2.3 Translasi Control Flow

Tantangan terbesar pada pembentukan Intermediate Code adalah meratakan struktur *control flow* bersarang (`if-else`, `while`) menjadi daftar linier. Solusinya menggunakan kombinasi **Label** dan **Jump**:

Label adalah penanda statis posisi suatu baris dalam TAC. **Jump mutlak** (JMP) melompat tanpa syarat ke sebuah label, sedangkan **jump bersyarat** (JPC) melompat hanya jika kondisi yang dievaluasi bernilai *false*. Konvensi melompat-jika-salah ini disebut *IF_FALSE* dan memanfaatkan prinsip *fall-through*: ketika kondisi benar, eksekusi langsung berlanjut ke instruksi berikutnya tanpa lompatan, sehingga alur kontrol menjadi lebih sederhana. Pada tugas besar ini konvensi *IF_FALSE* bersifat wajib.

Berikut pola translasi `if-else` (kiri) dan `while` (kanan) menjadi Intermediate Code.

IF-ELSE

```
t1 := x > 10
JPC t1 -> L_ELSE
y := 1
JMP L_END
L_ELSE:
  y := 0
L_END:
```

WHILE

```
L_START:
  t1 := i < 5
  JPC t1 -> L_END
  i := i + 1
  JMP L_START
L_END:
```

1.3 Katalog Instruksi Intermediate Code

Intermediate Code Arion menggunakan himpunan instruksi bergaya ORKOM. Tiap instruksi terdiri atas *mnemonic* dan operand opsional. Tabel 1 merangkum instruksi yang didukung, sedangkan Tabel 2 merinci operasi yang dipanggil melalui instruksi OPR.

Tabel 1: Daftar instruksi Intermediate Code.

Instruksi	Operasi	Keterangan
LIT <i>v</i>	Load Literal	Memasukkan nilai literal <i>v</i> ke stack.
LOD <i>a</i>	Load Value	Memuat nilai dari alamat <i>a</i> ke stack.
STO <i>a</i>	Store Value	Mengambil nilai puncak stack, menyimpannya ke alamat <i>a</i> .
CAL <i>l</i>	Call	Memanggil fungsi/prosedur pada baris <i>l</i> .
INT <i>m</i>	Initiate Memory	Mengalokasikan memori (stack frame) berukuran <i>m</i> .
JMP <i>l</i>	Unconditional Jump	Lompat ke baris <i>l</i> tanpa syarat.
JPC <i>l</i>	Conditional Jump	Lompat ke baris <i>l</i> bila kondisi puncak stack salah.
OPR <i>o</i>	Operation	Menjalankan operasi bernomor <i>o</i> (lihat Tabel 2).
RET	Return	Keluar dari fungsi/prosedur atau mengakhiri program.

Tabel 2: Daftar operasi instruksi OPR.

No	Nama	Fungsi	No	Nama	Fungsi
1	NEG	Negasi puncak stack	8	NEQ	Tidak sama dengan
2	ADD	Penjumlahan	9	LSS	Kurang dari
3	SUB	Pengurangan	10	GEQ	Lebih dari sama dengan
4	MUL	Perkalian	11	GTR	Lebih dari
5	DIV	Pembagian	12	LEQ	Kurang dari sama dengan
6	MOD	Modulus	13	WRT	Cetak output
7	EQL	Sama dengan	14	WRTLN	Cetak output + newline

1.4 Arsitektur Mesin Eksekusi: Interpreter dan Stack

1.4.1 Virtual Machine dan Siklus Eksekusi

Interpreter pada Milestone 4 berperan sebagai sebuah *Virtual Machine* (VM). Sama seperti CPU sungguhan, VM membaca instruksi satu per satu menggunakan penunjuk *Instruction Pointer* (IP) atau *Program Counter*. VM bekerja dalam **siklus eksekusi** tiga tahap: **Fetch** (membaca instruksi yang ditunjuk IP), **Decode** (membedah mnemonic dan operand-nya), dan **Execute** (melakukan aksi nyata). Setelah Execute, IP bergerak maju ke instruksi berikutnya, kecuali pada instruksi lompat (JMP/JPC) yang menggeser IP langsung ke baris tujuan. Siklus berulang hingga IP mencapai akhir program atau menemui RET.

1.4.2 Memori Eksekusi: The Stack

Struktur memori utama VM adalah **stack** yang bekerja dengan prinsip **LIFO** (*Last In, First Out*)—data hanya bisa dimasukkan (*push*) dan diambil (*pop*) dari puncak. Stack dipilih, bukan array biasa, karena berkaitan erat dengan *scope* dan pemanggilan fungsi. Setiap kali program memasuki blok atau memanggil fungsi, VM membuat *stack frame* baru. Pada konvensi yang dipakai, tiga sel pertama setiap frame selalu terisi **Static Link** (alamat 0), **Dynamic Link** (alamat 1), dan **Return Address** (alamat 2); sel selanjutnya (mulai alamat 3) digunakan untuk variabel. Akses variabel dilakukan relatif terhadap *Base Pointer* (BP) frame aktif, sementara *Stack Pointer* (SP) menunjuk puncak stack.

puncak – operand/hasil sementara	← SP (puncak stack)
alamat 4 – Variable (y)	
alamat 3 – Variable (x)	
alamat 2 – Return Address	
alamat 1 – Dynamic Link	
alamat 0 – Static Link	← BP (basis frame)

Gambar 2: Struktur stack frame: tiga sel administratif diikuti sel variabel; ruang di atasnya dipakai sebagai operand dan hasil sementara perhitungan.

1.5 Keamanan Runtime dan Penanganan Kerentanan

Interpreter yang tangguh tidak hanya menjalankan kode yang benar, tetapi juga mampu bertahan (tidak *crash*) ketika diberi kode yang salah atau tidak masuk akal. Berbeda dengan bahasa tingkat rendah yang menyerahkan keamanan kepada *programmer*, bahasa yang berjalan di atas VM wajib memiliki mekanisme perlindungan internal. Sebagai bagian dari spesifikasi (termasuk tantangan bonus), interpreter Arion mendeteksi dan menangani sejumlah kerentanan berikut.

Kerentanan pada stack. *Stack overflow* terjadi ketika frame terus ditambahkan melampaui batas memori, umumnya akibat rekursi tanpa *base case*; interpreter membatasi kedalaman frame dan melempar error bila terlampaui. *Stack underflow* terjadi ketika operasi *pop* dipanggil saat stack kosong. *Stack smashing* adalah perusakan *return address* oleh data yang meluber, dideteksi menggunakan *canary* pada sel return address. *Stack corruption* terjadi ketika jumlah *push* dan *pop* tidak simetris sehingga indeks frame bergeser; ini dideteksi dengan memverifikasi posisi stack pointer saat keluar dari frame.

Kerentanan memori dan alur. *Out-of-Bounds Access* dicegah dengan memvalidasi setiap alamat pada LOD/STO terhadap batas memori. *Invalid Jump Target* dicegah dengan memverifikasi target JMP, JPC, dan CAL berada dalam rentang program sebelum IP dipindahkan.

Kerentanan aritmatika. Tipe *integer* memiliki kapasitas terbatas; operasi yang melampaui batas atas (*overflow*) atau bawah (*underflow*) akan menyebabkan *wrap-around* yang mengacaukan perhitungan. Interpreter mendeteksi kondisi ini sebelum hasil sempat membungkus, lalu

melempar **OverflowError** atau **UnderflowError**. Demikian pula pembagian dengan nol dicegat sebagai **DivisionByZeroError**.

Runtime type checking. Karena Arion bersifat *strongly-typed* dan sebagian besar tipe telah diverifikasi pada analisis semantik, runtime type checking tidak diwajibkan. Meski demikian, interpreter tetap memvalidasi bahwa operand operasi aritmatika benar-benar bernilai numerik saat eksekusi; bila tidak, ia melempar **TypeMismatchError** alih-alih melakukan konversi diam-diam.

2. Perancangan dan Implementasi

2.1 Perancangan

2.1.1 Gambaran Umum Sistem

Pada Milestone 4, program menambahkan dua tahap baru pada pipeline kompiler: *Intermediate Code Generation* dan *Execution*. Alur kerjanya dapat diringkas sebagai berikut.

Decorated AST → Intermediate Code (TAC) → Stack Machine → Program
Output

Generator menerima Decorated AST hasil Milestone 3 lalu menelusurinya untuk menghasilkan daftar instruksi TAC. Daftar tersebut kemudian dikonversi melalui sebuah *adapter* dan dieksekusi oleh stack machine, yang mengelola memori dalam bentuk stack dan mencetak keluaran nyata ketika menemui instruksi WRT/WRTLN. Spesifikasi prosesnya: **masukan** berupa Decorated AST, **keluaran** berupa Intermediate Code beserta output program, dengan **algoritma** penelusuran berbasis DFS untuk pembentukan TAC dan *stack machine* untuk eksekusi.

2.1.2 Teknologi yang Digunakan

Sesuai ketentuan, bahasa pemrograman yang digunakan disamakan dengan milestone sebelumnya, yaitu C++ dengan standar C++17, dikompilasi menggunakan g++ dan diotomasi dengan GNU Make. Tidak ada pustaka eksternal yang digunakan; seluruh komponen dibangun di atas pustaka standar C++ (<vector>, <string>, <unordered_map>, <sstream>, <memory>). Deteksi *overflow* aritmatika memanfaatkan builtin `__builtin_*_overflow` milik g++.

2.1.3 Struktur Modul dan Arsitektur File

Sesuai prinsip modularitas, kode Milestone 4 dipisahkan berdasarkan tanggung jawab ke dalam dua direktori: `src/intermediate/` (pembentukan TAC) dan `src/interpreter/` (eksekusi). Tabel 3 memerinci berkas-berkas baru beserta perannya.

Tabel 3: Arsitektur berkas Milestone 4.

Berkas	Tanggung Jawab
<code>intermediate/tac.{hpp,cpp}</code>	Kelas <code>TacGenerator</code> : konversi Decorated AST $\rightarrow$ TAC.
<code>interpreter/runtime_value.*</code>	Struktur <code>RuntimeValue</code> : nilai bertipe pada stack.
<code>interpreter/runtime_instruction.hpp</code>	Struktur <code>RuntimeInstruction</code> : satu instruksi siap eksekusi.
<code>interpreter/runtime_context.*</code>	Struktur <code>RuntimeContext</code> : stack, IP, BP, dan pembukuan frame.
<code>interpreter/stack_machine.*</code>	Kelas <code>StackMachine</code> : inti siklus fetch–decode–execute.
<code>interpreter/runtime_error_hook.*</code>	<code>RuntimeErrorHook</code> : klasifikasi dan pelemparan error runtime.
<code>interpreter/tac_adapter.*</code>	<code>TacAdapter</code> : jembatan <code>TacInstr</code> $\rightarrow$ <code>RuntimeInstruction</code> .
<code>interpreter/interpreter.*</code>	Kelas <code>Interpreter</code> : pembungkus VM + penangkap error.

2.1.4 Justifikasi Pilihan Implementasi

Penelusuran berbasis DFS. Pembentukan TAC dilakukan dengan menelusuri AST secara *depth-first*. Untuk ekspresi, penelusuran postorder menjamin operand sudah berada di stack sebelum operatornya dijalankan; untuk pernyataan, penelusuran preorder yang dipadu pembentukan label menjaga urutan eksekusi sesuai struktur program.

Stack machine, bukan register machine. Model stack machine dipilih karena selaras dengan konvensi instruksi yang disediakan (gaya ORKOM) dan menghapus kebutuhan akan alokasi register/variabel sementara eksplisit—hasil antara cukup ditinggalkan pada puncak stack.

Pola Adapter. `TacGenerator` (tim pembentuk kode) dan `StackMachine` (tim interpreter) sengaja dipisahkan oleh `TacAdapter`. Dengan begitu, kode interpreter tidak bergantung pada detail internal generator; perubahan pada salah satu sisi tidak merembet ke sisi lain.

Error sebagai pengecualian, bukan crash. Seluruh pelanggaran runtime dilempar sebagai `RuntimeError` dan ditangkap di satu titik (`Interpreter::run`), sehingga program selalu berhenti secara *graceful* dengan pesan informatif alih-alih *segmentation fault*.

2.2 Implementasi Intermediate Code Generator

`TacGenerator` mengubah Decorated AST menjadi vektor `TacInstr`. Tiap instruksi merekam *opcode*, daftar argumen, label opsional, serta target lompat yang terisi setelah resolusi.

2.2.1 Pemetaan Alamat Memori

Sebelum kode dihasilkan, seluruh deklarasi variabel dipindai dan dipetakan ke alamat memori mulai dari 3 (sel 0–2 dicadangkan untuk static link, dynamic link, dan return address). Ukuran memori frame utama lalu dialokasikan dengan instruksi INT sebesar $3 + (\text{jumlah variabel})$.

Listing 1: Pemetaan alamat variabel dan alokasi memori awal.

```

1 void TacGenerator::collectDecls(const AstNodePtr& program) {
2     for (auto& child : program->children) {
3         if (child->kind != AstKind::Declarations) continue;
4         for (auto& decl : child->children) {
5             if (decl->kind == AstKind::VarDecl && !decl->value.empty()) {
6                 string key = lower(decl->value);
7                 if (addrMap.find(key) == addrMap.end())
8                     addrMap[key] = nextAddr++; // 3, 4, 5, ...
9             }
10        }
11    }
12 }
13 // ... pada generate():
14 int varCount = static_cast<int>(addrMap.size());
15 emit("INT", {"0", to_string(3 + varCount)}); // INT 0 <3 + jml var>

```

2.2.2 Pembentukan Ekspresi (DFS Postorder)

Ekspresi dibentuk secara rekursif. Literal menjadi LIT, variabel menjadi LOD pada alamatnya, sedangkan operasi biner mem-*push* operand kiri lalu kanan sebelum memanggil OPR dengan opcode yang sesuai. Pemetaan operator ke opcode mengikuti Tabel 2.

Listing 2: Pembentukan ekspresi: hasil ditinggalkan di puncak stack.

```

1 void TacGenerator::genExpr(const AstNodePtr& node) {
2     switch (node->kind) {
3         case AstKind::Number: case AstKind::RealNumber:
4         case AstKind::BoolLit: /* ... */
5             emit("LIT", {"0", node->value}); break;
6         case AstKind::Var:
7             emit("LOD", {"0", to_string(addrOf(node))}); break;
8         case AstKind::BinOp: {
9             genExpr(node->children[0]); // operand kiri di-push lebih dulu
10            genExpr(node->children[1]); // lalu operand kanan
11            int code = oprCode(node->value);
12            if (code >= 0) emit("OPR", {"0", to_string(code)});
13            break;
14        }
15         case AstKind::UnaryOp:
16             genExpr(node->children[0]);
17             if (node->value == "-") emit("OPR", {"0", "1"}); // NEG
18             break;
19     }
20 }

```

2.2.3 Pembentukan Control Flow

Pernyataan `if` dan `while` dibentuk dengan menyemai label simbolik (`L1`, `L2`, ...) yang nantinya diresolusi menjadi nomor baris. Konvensi *IF_FALSE* diwujudkan oleh JPC: kondisi di-*push*, lalu JPC melompati blok bila kondisi salah.

Listing 3: Pembentukan `if-else` dan `while` dengan label & jump.

```

1 void TacGenerator::genIf(const AstNodePtr& node) {
2     string elseLbl = newLabel(), endLbl = newLabel();
3     genExpr(cond); // push kondisi
4     emit("JPC", {"0", elseLbl}); // salah -> lompat ke else
5     if (thenNode) genNode(thenNode);
6     if (elseNode) emit("JMP", {"0", endLbl});
7     emitLabel(elseLbl);
8     if (elseNode) genNode(elseNode);
9     emitLabel(endLbl);
10 }
11
12 void TacGenerator::genWhile(const AstNodePtr& node) {
13     string startLbl = newLabel(), endLbl = newLabel();
14     emitLabel(startLbl);
15     genExpr(cond); // push kondisi
16     emit("JPC", {"0", endLbl}); // salah -> keluar loop
17     if (body) genNode(body);
18     emit("JMP", {"0", startLbl}); // ulangi
19     emitLabel(endLbl);
20 }

```

2.2.4 Resolusi Label dan Pencetakan

Setelah seluruh kode terbentuk, label diresolusi menjadi nomor baris 1-based, lalu *pseudo-op* LABEL dihapus agar daftar instruksi padat dan langsung dapat dieksekusi (jika dibiarkan, label akan menggeser indeks setiap instruksi sesudahnya). Saat dicetak, JMP/JPC ditampilkan dengan target numeriknya dalam format kanonik `<baris> <OP> <lev> <operand>`.

2.3 Implementasi Interpreter (Stack Machine)

2.3.1 Representasi Nilai dan Konteks

Setiap sel stack menyimpan `RuntimeValue` yang bertipe (`INTEGER`, `REAL`, `BOOLEAN`, `CHAR`, `STRING`, atau `NONE`). Tipe yang melekat inilah yang memungkinkan *runtime type checking* dan pencetakan keluaran yang benar. Seluruh keadaan eksekusi disimpan dalam `RuntimeContext`: vektor `stack`, base pointer `bp`, return address `ra`, dynamic link `dl`, kedalaman frame, instruction pointer `ip`, vektor program, serta *buffer* keluaran.

2.3.2 Siklus Fetch–Decode–Execute

Metode `run()` mengulang `step()` hingga program *halt* atau IP melewati akhir program. Tiap `step()` membaca instruksi pada IP lalu mendispatch ke *handler* sesuai opcode.

Listing 4: Inti siklus eksekusi dan dispatch instruksi.

```

1 void StackMachine::run() {
2     while (!context.halted && context.ip < context.program.size())
3         step();
4 }
5 void StackMachine::execute(const RuntimeInstruction& instr) {
6     const string& op = instr.op;
7     if (op == "INT") { execINT(instr); return; }
8     if (op == "LIT") { execLIT(instr); return; }
9     if (op == "LOD") { execLOD(instr); return; }
10    if (op == "STO") { execSTO(instr); return; }
11    if (op == "JMP") { execJMP(instr); return; }
12    if (op == "JPC") { execJPC(instr); return; }
13    if (op == "OPR") { execOPR(instr); return; }
14    if (op == "RET") { execRET(instr); return; }
15    // ... CAL, LABEL ...
16    RuntimeErrorHook::invalidOperand(op, "unknown instruction");
17 }

```

2.3.3 Inisiasi Frame dan Akses Memori

Instruksi INT membentuk frame baru: mendorong static link, dynamic link, dan return address, lalu mengalokasikan sel variabel. Akses variabel (LOD/STO) dihitung relatif terhadap bp dan divalidasi terhadap batas stack sebelum digunakan.

Listing 5: LOD dengan validasi akses memori (mencegah OOB).

```

1 void StackMachine::execLOD(const RuntimeInstruction& instr) {
2     int addr = parseIntArg(instr, 1);
3     int absIdx = context.bp + addr;
4     if (absIdx < 0 || (size_t)absIdx >= context.stack.size())
5         RuntimeErrorHook::invalidMemoryAccess(addr, context.bp,
6                                                 context.stack.size());
7     push(context.stack[absIdx]);
8     context.ip++;
9 }

```

2.3.4 Lompatan dan Konvensi IF_FALSE

JPC mengeluarkan kondisi dari puncak stack; bila bernilai *false*, IP dipindahkan ke target, jika tidak IP maju satu langkah. Setiap target lompat divalidasi oleh `resolveJump` agar tidak menunjuk ke luar program.

Listing 6: JPC (jump-if-false) dan validasi target lompat.

```

1 void StackMachine::execJPC(const RuntimeInstruction& instr) {
2     RuntimeValue cond = pop();
3     if (!cond.isTruthy()) context.ip = resolveJump(instr.target);
4     else context.ip++;
5 }
6 size_t StackMachine::resolveJump(int target) const {

```

```

7     int n = (int)context.program.size();
8     if (target < 1 || target > n)
9         RuntimeErrorHook::invalidJumpTarget(target, n);
10    return (size_t)(target - 1);    // 1-based -> 0-based
11 }

```

2.3.5 Operasi melalui OPR

`execOPR` mendispatch nomor operasi ke sub-handler. Operasi aritmatika mengeluarkan dua operand (atau satu untuk `NEG`), memvalidasi tipenya, melakukan perhitungan ber-*checked* terhadap overflow, lalu mem-*push* hasilnya. Operasi perbandingan menghasilkan `BOOLEAN`, sedangkan `WRT`/`WRTLN` menulis nilai puncak stack ke *buffer* keluaran.

2.4 Implementasi Error Handling dan Keamanan Runtime

Seluruh kondisi error dipusatkan pada modul `RuntimeErrorHook`. Setiap pelanggaran memanggil fungsi yang memformat pesan `[ERROR] <Jenis>: <detail> !` lalu melempar `RuntimeError` dengan klasifikasi jenisnya, sehingga pesan konsisten dan mudah ditelusuri.

Aritmatika ber-checked. Operasi integer memakai *builtin* `g++` untuk mendeteksi *wrap-around*, lalu melempar `OverflowError`/`UnderflowError`.

Listing 7: Penjumlahan dengan deteksi overflow/underflow.

```

1 long long StackMachine::addChecked(long long a, long long b, const string& op
  ) {
2     long long r;
3     if (__builtin_add_overflow(a, b, &r)) {
4         if (a > 0 && b > 0) RuntimeErrorHook::numericOverflow(op);
5         RuntimeErrorHook::numericUnderflow(op);
6     }
7     return r;
8 }

```

Deteksi stack corruption dan smashing. Saat `INT`, sebuah `FrameGuard` mencatat posisi puncak yang seharusnya saat keluar (*expected top*) dan salinan return address sebagai *canary*. Pada `RET`, posisi puncak aktual dibandingkan dengan *expected top* (deteksi *corruption*) dan sel return address dibandingkan dengan *canary* (deteksi *smashing*).

Listing 8: Verifikasi keseimbangan frame saat `RET`.

```

1 const FrameGuard guard = context.frames.back();
2 context.frames.pop_back();
3 int actualTop = (int)context.stack.size();
4 if (actualTop != guard.expectedTop)    // push/pop tak simetris
5     RuntimeErrorHook::stackCorruption("unbalanced push/pop ...",
6                                       guard.expectedTop, actualTop);
7 int raIdx = guard.basePtr + 2;    // sel return address
8 if (context.stack[raIdx].intVal != guard.canaryRa)
9     RuntimeErrorHook::stackSmashing(guard.canaryRa, context.stack[raIdx].
    intVal);

```

Tabel 4 merangkum jenis-jenis error runtime yang ditangani beserta mekanisme deteksinya.

Tabel 4: Ringkasan penanganan error runtime.

Jenis Error	Mekanisme Deteksi
DivisionByZeroError	Pembagi bernilai nol pada DIV/MOD.
OverflowError / UnderflowError	Hasil aritmatika integer membungkus (<i>wrap-around</i>).
StackOverflowError	Kedalaman frame atau ukuran stack melampaui batas.
StackUnderflowError	<i>Pop</i> dipanggil saat stack kekurangan nilai.
InvalidJumpError	Target JMP/JPC/CAL di luar program.
InvalidMemoryAccessError	Alamat LOD/STO di luar batas stack.
StackCorruptionError	Posisi stack pointer tidak sesuai saat keluar frame.
StackSmashingError	<i>Canary</i> return address tertimpa data lain.
TypeMismatchError	Operand aritmatika ternyata bukan nilai numerik.
InvalidOperandError	Argumen instruksi hilang atau bukan integer valid.

3. Pengujian

Bagian ini menyajikan enam kasus uji unik yang mencakup penugasan, ekspresi aritmatika dan relasional, ekspresi bersarang, alur kontrol, serta dua kasus penanganan error runtime. Setiap kasus menampilkan kode sumber Arion sebagai masukan, Intermediate Code yang dihasilkan, dan keluaran nyata program. Seluruh hasil di bawah diperoleh dengan menjalankan kompiler pada berkas uji yang bersangkutan; bukti tangkapan layar disertakan sebagai placeholder.

3.1 Kasus 1 – Penugasan dan Cetak Literal

Tujuan. Memverifikasi alokasi memori (INT), penugasan literal ke variabel (LIT+STO), pembacaan kembali (LOD), dan penyimpanan nilai variabel.

Masukan (src_assignment.txt):

```
program SimpleAssign;
var
  n: integer;
begin
  n := 10;
  writeln(n);
  n := 99;
  writeln(n);
end.
```

Intermediate Code:

```
1 INT 0 4
2 LIT 0 10
3 STO 0 3
4 LOD 0 3
5 OPR 0 14
6 LIT 0 99
7 STO 0 3
8 LOD 0 3
9 OPR 0 14
10 RET 0 0
```

Keluaran Program:

10
99

[PLACEHOLDER SCREENSHOT KASUS 1]
Tangkapan layar Intermediate Code dan Program Output untuk SimpleAssign.

Analisis. Variabel `n` dipetakan ke alamat 3 sehingga INT mengalokasikan 4 sel (3 administratif + 1 variabel). Penimpaan `n := 99` menyimpan ulang ke alamat yang sama. Keluaran sesuai harapan.

3.2 Kasus 2 – Ekspresi Aritmatika dan Relasional

Tujuan. Menguji operator aritmatika (+, -, *, div), prioritas tanda kurung, serta operator relasional yang menghasilkan nilai `boolean`.

Masukan (`src_expr_assign.txt`):

```
program ExprAssign;
var
  a, b, c: integer;
begin
  a := 6 + 4;
  b := (a - 2) * 3;
  c := b div 5;
  writeln(a);
  writeln(b);
  writeln(c);
  writeln(a = 10);
  writeln(b > c);
end.
```

Intermediate Code:

1 INT 0 6	16 LOD 0 3
2 LIT 0 6	17 OPR 0 14
3 LIT 0 4	18 LOD 0 4
4 OPR 0 2	19 OPR 0 14
5 ST0 0 3	20 LOD 0 5
6 LOD 0 3	21 OPR 0 14
7 LIT 0 2	22 LOD 0 3
8 OPR 0 3	23 LIT 0 10
9 LIT 0 3	24 OPR 0 7
10 OPR 0 4	25 OPR 0 14
11 ST0 0 4	26 LOD 0 4
12 LOD 0 4	27 LOD 0 5
13 LIT 0 5	28 OPR 0 11
14 OPR 0 5	29 OPR 0 14
15 ST0 0 5	30 RET 0 0

Keluaran Program:

10
24
4
TRUE
TRUE

[PLACEHOLDER SCREENSHOT KASUS 2]

Tangkapan layar Intermediate Code dan Program Output untuk ExprAssign.

Analisis. Perhitungan terverifikasi: $a = 10$, $b = (10 - 2) \times 3 = 24$, $c = 24 \div 5 = 4$ (pembagian integer). Operator relasional $=$ (OPR 7) dan $>$ (OPR 11) menghasilkan **boolean** yang dicetak sebagai **TRUE**.

3.3 Kasus 3 – Ekspresi Bersarang dan Modulus

Tujuan. Memastikan penelusuran DFS meratakan ekspresi bersarang $(a + b) * (c - d)$ dengan benar—hasil antara ditinggalkan di puncak stack sebagai pengganti variabel sementara—serta menguji operator **mod**.

Masukan (src_nested_expr.txt):

```
program NestedExpr;
var
  a, b, c, d, x: integer;
begin
  a := 3;  b := 4;
  c := 10; d := 2;
  x := (a + b) * (c - d);
  writeln(x);
  writeln(c mod d);
end.
```

Intermediate Code (ringkas):

1 INT 0 8	13 LOD 0 5
2 LIT 0 3	14 LOD 0 6
3 ST0 0 3	15 OPR 0 3 ; c
- d	
...	16 OPR 0 4 ;
(..)*(..)	
10 LOD 0 3	17 ST0 0 7
11 LOD 0 4	18 LOD 0 7
12 OPR 0 2	19 OPR 0 14 ;
writeln(x)	
; c - d	20 LOD 0 5
	21 LOD 0 6
	22 OPR 0 6 ; c
	mod d
	23 OPR 0 14
	24 RET 0 0

Keluaran Program:

```
56
0
```


[PLACEHOLDER SCREENSHOT KASUS 3]

Tangkapan layar Intermediate Code dan Program Output untuk NestedExpr.

Analisis. $x = (3 + 4) \times (10 - 2) = 7 \times 8 = 56$ dan $c \bmod d = 10 \bmod 2 = 0$. Subekspresi kiri dihitung lebih dulu (OPR 2), lalu subekspresi kanan (OPR 3), barulah keduanya dikalikan (OPR 4)—membuktikan urutan evaluasi postorder yang benar tanpa membutuhkan nama **temporary** eksplisit.

3.4 Kasus 4 – Kontrol Alur IF-ELSE dan WHILE

Tujuan. Menguji translasi percabangan dan perulangan—konvensi *IF_FALSE* pada JPC, lompat-mundur JMP pada **while**. Kasus ini merupakan contoh kanonik pada spesifikasi.

Masukan (src_testkeywords.txt):

```
program TestKeywords;
var
  x, y: integer;
begin
  x := 10;
  if x > 5 then
    y := x * 2
  else
    y := x div 2;
  writeln(y);
  while y > 0 do
  begin
    y := y - 1;
    writeln(y);
  end;
end.
```

Intermediate Code:

1 INT 0 5	16 ST0 0 4
2 LIT 0 10	17 LOD 0 4
3 ST0 0 3	18 OPR 0 14
4 LOD 0 3	19 LOD 0 4
5 LIT 0 5	20 LIT 0 0
6 OPR 0 11	21 OPR 0 11
7 JPC 0 13	22 JPC 0 30
8 LOD 0 3	23 LOD 0 4
9 LIT 0 2	24 LIT 0 1
10 OPR 0 4	25 OPR 0 3
11 ST0 0 4	26 ST0 0 4
12 JMP 0 17	27 LOD 0 4
13 LOD 0 3	28 OPR 0 14
14 LIT 0 2	29 JMP 0 19
15 OPR 0 5	30 RET 0 0

Keluaran Program: cabang **then** terpilih ($x = 10 > 5$) sehingga $y = 20$, lalu perulangan menghitung mundur hingga 0.

20	19	18	17	16	15	14	13	12	11	10
9	8	7	6	5	4	3	2	1	0	

(keluaran sebenarnya satu nilai per baris; ditampilkan dua baris di sini demi keringkasan).

[PLACEHOLDER SCREENSHOT KASUS 4]

Tangkapan layar Intermediate Code dan Program Output untuk TestKeywords.

Analisis. Intermediate Code yang dihasilkan identik dengan contoh pada spesifikasi. JPC 0 13 melompati blok **then** bila kondisi salah, JMP 0 17 melewati blok **else**, dan JMP 0 19 pada perulangan kembali ke evaluasi kondisi—membuktikan translasi control flow tepat.

3.5 Kasus 5 – Penanganan Error: Pembagian dengan Nol

Tujuan. Memverifikasi bahwa kesalahan runtime tidak membuat program *crash*, melainkan ditangkap dan dilaporkan secara *graceful*.

Masukan (src_divzero.txt):

```
program DivByZero;
var
  a, b: integer;
begin
  a := 10;
  b := 0;
  writeln(a div b);
end.
```

Intermediate Code:

```
1 INT 0 5
2 LIT 0 10
3 ST0 0 3
4 LIT 0 0
5 ST0 0 4
6 LOD 0 3
7 LOD 0 4
8 OPR 0 5      ; a div b
9 OPR 0 14
10 RET 0 0
```

Keluaran Program:

```
[ERROR] DivisionByZeroError: division by zero !
```

[PLACEHOLDER SCREENSHOT KASUS 5]

Tangkapan layar pesan DivisionByZeroError saat eksekusi src_divzero.

Analisis. Saat DIV (OPR 5) mendeteksi pembagi bernilai nol, `RuntimeErrorHook::divisionByZero()` melempar `RuntimeError` yang ditangkap `Interpreter::run()`. Program berhenti dengan pesan jelas, bukan *segmentation fault*.

3.6 Kasus 6 – Penanganan Error: Numerical Overflow

Tujuan. Menguji deteksi *integer overflow* pada operasi aritmatika sebelum nilai sempat membungkus (*wrap-around*).

Masukan (src_overflow.txt):

```
program NumericOverflow;
var
  big, hasil: integer;
begin
  big := 4000000000;
  hasil := big * big;
  writeln(hasil);
end.
```

Intermediate Code:

```
1 INT 0 5
2 LIT 0 4000000000
3 STO 0 3
4 LOD 0 3
5 LOD 0 3
6 OPR 0 4      ; big * big
7 STO 0 4
8 LOD 0 4
9 OPR 0 14
10 RET 0 0
```

Keluaran Program:

```
[ERROR] OverflowError: 'MUL' result exceeds the maximum integer value
      !
```

[PLACEHOLDER SCREENSHOT KASUS 6]
 Tangkapan layar pesan *OverflowError* saat eksekusi *src_overflow*.

Analisis. $4,000,000,000^2 = 1.6 \times 10^{19}$ melampaui batas maksimum integer. `mulChecked` mendeteksinya melalui `__builtin_mul_overflow` dan melempar `OverflowError`, sehingga hasil yang salah akibat *wrap-around* tidak pernah dicetak.

3.7 Ringkasan Pengujian

Tabel 5: Ringkasan hasil pengujian Milestone 4.

No	Aspek yang Diuji	Keluaran	Status
1	Penugasan & cetak literal	10, 99	Sesuai
2	Aritmatika & relasional	10, 24, 4, TRUE, TRUE	Sesuai
3	Ekspresi bersarang & modulus	56, 0	Sesuai
4	Control flow IF-ELSE & WHILE	20...0	Sesuai
5	Error: pembagian dengan nol	DivisionByZeroError	Sesuai
6	Error: numerical overflow	OverflowError	Sesuai

Keenam kasus uji memberikan hasil sesuai harapan. Generator menghasilkan Intermediate Code yang valid (identik dengan contoh spesifikasi pada Kasus 4), interpreter mengeksekusinya dengan benar, dan mekanisme keamanan runtime berhasil menangkap kondisi error tanpa menyebabkan program *crash*.

4. Kesimpulan dan Saran

4.1 Kesimpulan

Milestone 4 berhasil menyatukan seluruh komponen kompiler Arion yang dibangun pada milestone sebelumnya menjadi sebuah kompiler yang utuh dan dapat dieksekusi. Dari pengerjaan tahap ini dapat ditarik beberapa kesimpulan.

Pertama, *Decorated AST* hasil analisis semantik berhasil diratakan menjadi *Three-Address Code* melalui penelusuran berbasis DFS. Ekspresi diterjemahkan secara postorder sehingga operand selalu tersedia di stack sebelum operatornya dijalankan, sementara struktur control flow *if-else* dan *while* diratakan menggunakan kombinasi label dan instruksi lompat dengan konvensi *IF_FALSE*. Intermediate Code yang dihasilkan terbukti identik dengan contoh pada spesifikasi.

Kedua, interpreter berbasis *stack machine* mampu mengeksekusi Intermediate Code dengan benar melalui siklus *fetch-decode-execute*, mengelola memori dalam bentuk stack frame (static link, dynamic link, return address, dan variabel), serta menghasilkan keluaran nyata program melalui operasi WRT/WRTLN.

Ketiga, aspek keamanan runtime—termasuk tantangan bonus—telah diimplementasikan. Interpreter menangani pembagian dengan nol, overflow/underflow aritmatika, stack overflow/underflow, stack corruption, stack smashing, akses memori dan target lompat di luar batas, serta pemeriksaan tipe saat eksekusi. Seluruh kondisi error dilaporkan secara *graceful* dengan pesan informatif tanpa membuat program *crash*, sebagaimana ditunjukkan pada Kasus 5 dan 6.

Keempat, kode disusun secara modular dengan pemisahan tanggung jawab yang jelas antara pembentukan kode (*intermediate/*) dan eksekusi (*interpreter/*), dijembatani oleh pola *adapter* sehingga kedua sisi tetap terpisah (*loosely coupled*).

4.2 Saran

Beberapa hal dapat dikembangkan lebih lanjut. Implementasi pemanggilan fungsi/prosedur (CAL/RET dengan *nested scope* dan *passing* argumen penuh) masih bersifat dasar dan dapat dilengkapi agar mendukung rekursi serta variabel lokal antar-frame secara menyeluruh. Selain itu, pembentukan kode dapat diperluas untuk menangani tipe data terstruktur seperti *array* dan *record* beserta validasi indeks *out-of-bounds* saat runtime. Dari sisi optimasi, *constant folding* dan eliminasi instruksi redundan dapat ditambahkan pada tahap pembentukan TAC untuk menghasilkan kode yang lebih ringkas. Terakhir, cakupan pengujian otomatis dapat diperluas dengan kerangka uji regresi agar setiap perubahan dapat diverifikasi dengan cepat.

5. Lampiran

5.1 Tautan Release Repository GitHub

Kode sumber lengkap kompiler Arion beserta release Milestone 4 tersedia pada repositori GitHub berikut.

- **Repositori:** <https://github.com/Nusss0/CNA-Tubes-IF2224-2026>
- **Release Milestone 4 (tag v0.4.1):**
<https://github.com/Nusss0/CNA-Tubes-IF2224-2026/releases/tag/v0.4.1>

5.2 Pembagian Tugas

Pembagian tugas dan persentase kontribusi setiap anggota kelompok pada Milestone 4 dirangkum pada Tabel 6.

Tabel 6: Pembagian tugas dan persentase kontribusi Milestone 4.

Nama	NIM	Tugas Milestone 4	Kontribusi
Stevanus Agustaf Wongso	13524020	Validasi runtime pada stack machine, modul <code>RuntimeErrorHook</code> , deteksi overflow/underflow, stack corruption & smashing, runtime type checking (tantangan bonus).	25%
Renuno Yuqa Frinardi	13524080	Inti interpreter / stack machine (<code>StackMachine</code> , <code>RuntimeValue</code> , <code>RuntimeContext</code>), <code>TacAdapter</code> , dan penyusunan kasus uji Milestone 4.	25%
Valentino Daniel Kusumo	13524104	Pembentukan control flow pada generator (<code>genIf</code> , <code>genWhile</code>), integrasi pipeline pada <code>main</code> , serta penulisan laporan.	25%
Michael James Liman	13524106	Inti <code>TacGenerator</code> (pemetaan alamat, pembentukan ekspresi & penugasan, resolusi label, pencetakan), kasus uji ekspresi/control flow, dan penulisan laporan.	25%
Total			100%

6. Referensi

- [1] A. V. Aho, M. S. Lam, R. Sethi, dan J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, edisi ke-2. Boston: Addison-Wesley, 2006.
- [2] Tim Asisten IF2224, *Spesifikasi Milestone 4 – Tubes IF2224 TBFO: Intermediate Code and Interpreter*, Institut Teknologi Bandung, 2026.
- [3] Tim Asisten IF2224, *Lampiran Spesifikasi Milestone 4 – Guidebook Konvensi Intermediate Code*, Institut Teknologi Bandung, 2026.
- [4] P. Koopman, *Stack Computers: The New Wave*. [Daring]. Tersedia: https://users.ece.cmu.edu/~koopman/stack_computers/sec3_2.html
- [5] E. Bendersky, “Stack frame layout on x86-64,” 2011. [Daring]. Tersedia: <https://eli.thegreenplace.net/2011/09/06/stack-frame-layout-on-x86-64>
- [6] GEPL, “Decorated Abstract Syntax Tree,” University of Minho. [Daring]. Tersedia: https://epl.di.uminho.pt/~gepl/GEPL_DS/Alma2/dapast.php